

Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Тестирование программного обеспечения

Отчет по лабораторной работе №1

Вариант 95035

Выполнил:

Козлов Василий Сергеевич

Р3315

Проверила:

Харитонова Анастасия Евгеньевна

Санкт-Петербург, 2026

Содержание

1. Задание 1	3
2. Задание 2	4
3. Задание 3	7
4. Вывод	8

1. Задание 1

«Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.»

$$\arctan x = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{2n+1}, |x| \leq 1$$

Аргумент	Результат	Класс результата
0	0	валидный
0.1	0.0996686	валидный
0.5	0.463647609	валидный
-0.5	-0.463647609	валидный
1	0.785398163	валидный
-1	-0.785398163	валидный
1.5	ValueError	невалидный
-1.5	ValueError	невалидный

Таблица 1. Тестовые сценарии

```
package org.vaskozlov.lab1;

public class TaylorArctan {
    public static double taylorArctan(double x, int nTerms) {
        if (Math.abs(x) > 1.0) {
            throw new IllegalArgumentException(
                "Taylor series for arctan(x) converges only for |x| <=
1");
        }

        double sum = 0.0;
        double sign = 1.0;

        for (int k = 0; k < nTerms; k++) {
            double term = sign * Math.pow(x, 2 * k + 1) / (2 * k + 1);
            sum += term;
            sign = -sign;
        }

        return sum;
    }

    public static double taylorArctan(double x) {
        return taylorArctan(x, 10);
    }
}
```

2. Задание 2

«Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.»

```
package org.vaskozlov.lab1;

import java.util.*;
import java.util.stream.Collectors;

public class MergeSortWithTrace {
    public static List<Integer> sort(int[] arr, List<String> trace) {
        if (trace == null) {
            trace = new ArrayList<>();
        } else {
            trace.clear();
        }

        List<Integer> list =
Arrays.stream(arr).boxed().collect(Collectors.toList());
        return mergeSortRecursive(list, trace, "");
    }

    private static List<Integer> mergeSortRecursive(List<Integer> arr,
                                                    List<String> trace,
                                                    String prefix) {

        int n = arr.size();
        String current = appendCurrentArrayToTrace(arr, prefix);

        trace.add("ENTER " + current + " (len=" + n + ")");

        if (n <= 1) {
            trace.add("BASE_CASE_RETURN " + current);
            return new ArrayList<>(arr);
        }

        int mid = n / 2;
        trace.add(String.format("SPLIT mid=%d left_len=%d, right_len=%d",
mid, mid, n - mid));

        List<Integer> left = mergeSortRecursive(
            new ArrayList<>(arr.subList(0, mid)), trace, prefix + " L ");
        trace.add("LEFT_DONE " + current + " -> " + left);

        List<Integer> right = mergeSortRecursive(
            new ArrayList<>(arr.subList(mid, n)), trace, prefix + " R ");

        trace.add("RIGHT_DONE " + current + " -> " + right);

        trace.add("START_MERGE " + left + " + " + right);
    }
}
```

```

        List<Integer> merged = merge(left, right, trace);
        trace.add("MERGE_RESULT " + merged);

        return merged;
    }

    private static List<Integer> merge(List<Integer> left,
                                       List<Integer> right,
                                       List<String> trace) {
        List<Integer> merged = new ArrayList<>();
        int i = 0, j = 0;

        while (i < left.size() && j < right.size()) {
            if (left.get(i) <= right.get(j)) {
                merged.add(left.get(i));
                trace.add(String.format("TAKE_LEFT %d (i=%d, j=%d)",
left.get(i), i, j));
                i++;
            } else {
                merged.add(right.get(j));
                trace.add(String.format("TAKE_RIGHT %d (i=%d, j=%d)",
right.get(j), i, j));
                j++;
            }
        }

        while (i < left.size()) {
            merged.add(left.get(i));
            trace.add("REMAIN_LEFT " + left.get(i));
            i++;
        }

        while (j < right.size()) {
            merged.add(right.get(j));
            trace.add("REMAIN_RIGHT " + right.get(j));
            j++;
        }

        return merged;
    }

    private static String appendCurrentArrayToTrace(List<Integer> arr, String
prefix) {
        if (arr.isEmpty()) {
            return prefix + "[]";
        }
        return prefix + "[" +
            arr.stream()
                .map(Object::toString)
                .collect(Collectors.joining(", ")) + "];";
    }

```

} }

3. Задание 3

«Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели»

У Форда в руке был стеклянный флакончик, в котором плавала, переливаясь, маленькая желтая рыбка. Артур смотрел на него, моргая глазами. Ему хотелось, чтобы здесь было что-нибудь простое и знакомое, за что можно было бы мысленно зацепиться. Он чувствовал бы себя увереннее, если бы рядом с нижним бельем дентрасси, скворншельскими матрацами и человеком с Бетельгейзе, держащим маленькую рыбку и предлагающим засунуть ее в ухо, он увидел, к примеру, пакет кукурузных хлопьев.

Сущность	Основные атрибуты	Возможные методы / поведение	Роль в домене
Форд	флакон в руке	предлагать засунуть её в ухо	человек с Бетельгейзе
Артур	глаза	смотреть, моргать глазами	наблюдатель
Флакончик	стеклянный, содержимое		контейнер
Рыбка	маленькая, жёлтая, плавает, переливается	плавать, переливаться	объект внутри флакончика
Ухо		засунуть в него рыбку	место, куда предлагают засунуть рыбку

Таблица 2. Доменная модель

№	Тестовый сценарий	Ожидаемый результат	Тип проверки
1	Создать экземпляр «Рыбка»	Создана рыбка, цвет «жёлтая», размер «маленькая», состояние «плавает, переливается»	Создание + атрибуты
2	Создать экземпляр «Флакончик»	Создан стеклянный флакончик	Создание + атрибуты
3	Поместить рыбку во флакончик	Флакончик содержит рыбку	Композиция
4	Создать экземпляр «Форд»	Форд создан, рука пустая	Начальное состояние
5	Форд берёт флакончик в руку	У Форда в руке находится флакончик	Действие + изменение состояния
6	Создать экземпляр «Артур»	Артур создан	Начальное состояние

Таблица 3. Базовые тесты создания сущностей

№	Тестовый сценарий	Ожидаемый результат	Тип проверки
7	Форд предлагает Артуру засунуть рыбку в ухо	Форд выполнил действие «предлагать засунуть её в ухо»	Действие
8	Артур смотрит на флакончик	Артур выполнил действие «смотреть»	Действие наблюдателя
9	Артур моргает глазами	Артур выполнил действие «моргать глазами»	Действие наблюдателя
10	Попытка предложить засунуть рыбку в ухо, когда у Форда нет флакончика	Действие не выполнено	Защитное условие

Таблица 4. Тесты ключевых взаимодействий

№	Тестовый сценарий	Ожидаемый результат	Тип проверки
11	Рыбка после создания	Состояние = «плавает, переливается»	Инвариант сущности
12	Флакончик без содержимого	Содержимое пустое	Начальное состояние
13	Флакончик после помещения рыбки	Содержимое = [рыбка]	Инвариант композиции
14	Артур после создания	Глаза активны / готовы к действиям «смотреть», «моргать»	Начальное состояние
15	Попытка поместить во флакончик что-то кроме рыбки	Действие запрещено	Инвариант содержимого

Таблица 5. Тесты состояний и инвариантов

4. Вывод

В ходе выполнения лабораторной работы №1 проведено модульное тестирование разложения функции $\arctan(x)$ в ряд Тейлора с достаточным тестовым покрытием (валидные значения, граничные случаи и исключения), выполнено тестирование алгоритма сортировки слиянием с трассировкой характерных точек выполнения, а также сформирована доменная модель по заданному тексту с полным тестовым покрытием (создание сущностей, взаимодействия, состояния и инварианты).